

Collections et itérateurs en Rust

1. Collections	1
1.1 Tableaux	1
1.1.1 Tableaux de taille fixe	1
1.1.2 Tableau de taille variable (vec<T>)	1
1.2 Chaînes de caractères	2
1.2.1 Chaînes de taille fixe	2
1.2.2. Chaînes de taille variable	2
1.3. Dictionnaires	2
1.3.1. Ensembles	2
1.3.2. Tableaux associatifs	3
2. Itérateurs	3
2.1 Itération sur une collection	3
2.1.1 Itération sur un tableau ou un ensemble	3
2.1.2 Itération sur un tableau associatif	3
2.2 Itération sur les mots d'une chaîne	3
2.2.1 Itération globale	3
2.2.2. Itération pas à pas	3
2.3 Opérateurs sur les itérateurs	4

1. Collections

1.1 Tableaux

1.1.1 Tableaux de taille fixe

Créer un tableau de taille fixe contenant 18, 19 et 20.	<pre>let may_array = [18, 19, 20];</pre>
Créer un tableau de taille fixe contenant 100 zéros.	<pre>let my_array = [0;100];</pre>

1.1.2 Tableau de taille variable (vec<T>)

Créer un tableau dynamique contenant 18, 19 et 20.	<pre>let my_vector = vec![18, 19, 20];</pre>
Créer un tableau dynamique contenant 100 zéros.	<pre>let my_vector = vec![0;100];</pre>
Ajouter une valeur à la fin du tableau my_vector.	<pre>my_vector.push(21);</pre>

1.2 Chaînes de caractères

1.2.1 Chaînes de taille fixe.

Créer une chaîne de taille fixe contenant "Bonjour"	<pre>let my_string = "Bonjour";</pre>
---	---------------------------------------

1.2.2. Chaînes de taille variable

Créer une chaîne de taille fixe contenant "Bonjour"	<pre>let mut my_string = String::from("Bonjour");</pre>
Créer une chaîne dynamique vide.	<pre>let mut my_string = String::new();</pre>
Supprimer les espaces aux extrémités de <code>my_string</code>	<pre>my_string.trim();</pre>
Chaîne résultant de la conversion de la variable <code>x</code> .	<pre>x.to_string();</pre>
Chaîne complexe fabriquée à partir de la variable <code>x</code>	<pre>format!("La valeur de x est {}", x);</pre>
Afficher la chaîne <code>my_string</code>	<pre>println!("{}", my_string);</pre>

1.3. Dictionnaires

1.3.1. Ensembles

	<pre>use std::collections::BTreeSet as Set;</pre>
Créer un ensemble <code>my_set</code> ,	<pre>let my_set = Set::new();</pre>
Insérer élément dans <code>my_set</code>	<pre>my_set.insert(element);</pre>
Décider si élément est présent dans <code>my_set</code>	<pre>my_set.contains(element);</pre>

1.3.2. Tableaux associatifs00.

	<pre>use std::collections::BTreeMap as Map;</pre>
Créer un tableau associatif my_map	<pre>let my_map = Map::new();</pre>
Insérer (key, value) dans my_map	<pre>my_map.insert(key, value);</pre>
Récupérer la valeur associée à key.	<pre>my_map.get(key); my_map.get_mut(key);</pre>

2. Itérateurs

2.1 Itération sur une collection

2.1.1 Itération sur un tableau ou un ensemble

Itération par référence	Itération par capture de propriété
<pre>for element in my_vector.iter() { ... }</pre>	<pre>for element in my_vector.into_iter() { ... }</pre>

2.1.2 Itération sur un tableau associatif

```
for (key, value) in my_map.iter() {  
    ...  
}
```

2.2 Itération sur les mots d'une chaîne 😊

2.2.1 Itération globale

```
for word in my_string.split_whitespace() (  
    ...  
)
```

2.2.2. Itération pas à pas

```
let mut words = my_string.split_whitespace();  
match words.next() {  
    None => ...,  
    Some(word) => ...,  
}
```

2.3 Opérateurs sur les itérateurs

- | | |
|---|--|
| <ul style="list-style-type: none">- Itérer sur le tableau my_vector- mettre au carré ses éléments- et créer le tableau résultant | <pre>my_vector.iter()
 .map(x x * x)
 .collect::<Vec<_>>()</pre> |
|---|--|